

C# Collections

C# collections are designed to store, manage, and manipulate data efficiently. These collections allow adding, removing, finding, and sorting elements. They are broadly classified into **non-generic** and **generic** collections.

Key Features of Collections

1. **Adding and Inserting Items:** Collections allow adding elements dynamically.
 2. **Removing Items:** Items can be removed easily without resizing.
 3. **Finding, Sorting, and Searching:** Collections support search and sorting functionalities.
-

Types of Collections

Non-Generic Collections (Older, Legacy)

- Found in `System.Collections` namespace.
- Can store any type (objects), but not type-safe.
- Require boxing/unboxing for value types.
- Used mainly for legacy code compatibility.

Examples:

- **ArrayList** → dynamic array (stores any type).
- **Hashtable** → key-value pairs (keys & values are objects). where keys are unique.
- **SortedList** → sorted key-value pairs.
- **Queue** → first-in, first-out (FIFO).
- **Stack** → last-in, first-out (LIFO).

Generic Collections (Modern, Type-Safe)

- Found in **System.Collections.Generic** namespace.
- Store data of a **specific type** → type-safe, no boxing/unboxing.
- Better **performance** and **compile-time safety**.
- Introduced in .NET 2.0.

Examples:

- **List<T>** → dynamic array (type-safe).
 - **Dictionary<TKey, TValue>** → key-value pairs.
 - **SortedList<TKey, TValue>** → sorted key-value pairs.
 - **SortedDictionary<TKey, TValue>** → Similar to **SortedList**, but internally uses a binary search tree.
 - **Queue<T>** → FIFO.
 - **Stack<T>** → LIFO.
 - **HashSet<T>** → unique elements only.
 - **LinkedList<T>** → doubly linked list.
-

Non-Generic Collections

1. ArrayList

- **Definition:** Stores elements of different types, dynamically resizes as needed.
- **Namespace:** [System.Collections](#).

Example:

```
using System.Collections;
```

```
ArrayList arrayList = new ArrayList();  
arrayList.Add(1);  
arrayList.Add("Hello");  
arrayList.Add(3.14);
```

```
foreach (var item in arrayList)  
{  
    Console.WriteLine(item);  
}
```

2. Hashtable

- **Definition:** Stores key-value pairs where keys are unique.
- **Namespace:** [System.Collections](#).

Example:

```
using System.Collections;
```

```
Hashtable hashtable = new Hashtable();  
hashtable.Add("Id", 101);  
hashtable.Add("Name", "John");
```

```
foreach (DictionaryEntry entry in hashtable)  
{  
    Console.WriteLine($"{entry.Key}: {entry.Value}");  
}
```

3. SortedList

- **Definition:** Stores key-value pairs in sorted order by keys.
- **Namespace:** [System.Collections](#).

Example:

```
using System.Collections;
```

```
SortedList sortedList = new SortedList();  
sortedList.Add(2, "Two");  
sortedList.Add(1, "One");
```

```
foreach (DictionaryEntry entry in sortedList)  
{  
    Console.WriteLine($"{entry.Key}: {entry.Value}");  
}
```

4. Stack

- **Definition:** Follows the Last In, First Out (LIFO) principle.
- **Namespace:** [System.Collections](#).

Example:

```
using System.Collections;
```

```
Stack stack = new Stack();  
stack.Push(1);  
stack.Push(2);
```

```
Console.WriteLine(stack.Pop()); // Output: 2  
Console.WriteLine(stack.Peek()); // Output: 1
```

5. Queue

- **Definition:** Follows the First In, First Out (FIFO) principle.
- **Namespace:** `System.Collections`.

Example:

```
using System.Collections;
```

```
Queue queue = new Queue();  
queue.Enqueue(1);  
queue.Enqueue(2);
```

```
Console.WriteLine(queue.Dequeue()); // Output: 1  
Console.WriteLine(queue.Peek());   // Output: 2
```

Generic Collections

1. List<T>

- **Definition:** Strongly typed dynamic array.
- **Namespace:** `System.Collections.Generic`
- **Key Points:** Allows indexing, dynamic resizing.

```
using System.Collections.Generic;
```

```
List<int> numbers = new List<int> { 1, 2, 3 };  
numbers.Add(4);
```

```
foreach (int number in numbers)  
{  
    Console.WriteLine(number);  
}
```

2. Dictionary<TKey, TValue>

- **Definition:** Stores unique keys with values.
- **Namespace:** `System.Collections.Generic`
- **Key Points:** Fast lookups (hashing).

```
Dictionary<int, string> dictionary = new Dictionary<int, string>();  
dictionary.Add(1, "One");  
dictionary.Add(2, "Two");
```

```
foreach (var pair in dictionary)  
{  
    Console.WriteLine($"{pair.Key}: {pair.Value}");  
}
```

3. SortedList<TKey, TValue>

- Definition: Like **Dictionary**, but keeps items sorted by key.
- Namespace: **System.Collections.Generic**
- Key Points: Uses array internally, better memory usage.

```
SortedList<int, string> sortedList = new SortedList<int, string>();
sortedList.Add(2, "Two");
sortedList.Add(1, "One");
```

```
foreach (var pair in sortedList)
{
    Console.WriteLine($"{pair.Key}: {pair.Value}");
}
```

4. SortedDictionary<TKey, TValue>

- Definition: Similar to **SortedList**, but internally uses binary search tree.
- Namespace: **System.Collections.Generic**
- Key Points: Better for frequent insertions/deletions.

```
SortedDictionary<int, string> sortedDict = new SortedDictionary<int,
string>();
sortedDict.Add(2, "Two");
sortedDict.Add(1, "One");
```

```
foreach (var pair in sortedDict)
{
    Console.WriteLine($"{pair.Key}: {pair.Value}");
}
```

5. Stack<T>

- Definition: LIFO (Last-In-First-Out).
- Namespace: **System.Collections.Generic**
- Key Points: Use **.Push()** to insert, **.Pop()** to remove.

```
Stack<string> stack = new Stack<string>();  
stack.Push("A");  
stack.Push("B");
```

```
Console.WriteLine(stack.Pop()); // Output: B
```

6. Queue<T>

- Definition: FIFO (First-In-First-Out).
- Namespace: **System.Collections.Generic**
- Key Points: Use **.Enqueue()** to insert, **.Dequeue()** to remove.

```
Queue<int> queue = new Queue<int>();  
queue.Enqueue(1);  
queue.Enqueue(2);
```

```
Console.WriteLine(queue.Dequeue()); // Output: 1
```

7. HashSet<T>

- Definition: Collection of unique elements only.
- Namespace: **System.Collections.Generic**
- Key Points: Fast lookups, prevents duplicates.

```
HashSet<int> set = new HashSet<int> { 1, 2, 2, 3 };
```



```
foreach (var item in set)
{
    Console.WriteLine(item); // 1, 2, 3
}
```

8. LinkedList<T>

- **Definition:** Doubly linked list.
- **Namespace:** **System.Collections.Generic**
- **Key Points:** Efficient insertions/removals at any position.

```
LinkedList<string> linkedList = new LinkedList<string>();
linkedList.AddLast("First");
linkedList.AddLast("Second");
linkedList.AddFirst("Zero");
```

```
foreach (var item in linkedList)
{
    Console.WriteLine(item);
}
```

Tuples

Tuples are data structures for grouping multiple values.

1. Value Tuple (C# 7.0+)

- **Definition:** Lightweight struct for grouping values.

Example:

```
var student = (Id: 1, Name: "Alice", Marks: 85.5);
Console.WriteLine(student.Name); // Output: Alice
```

2. Reference Tuple (System.Tuple)

- **Definition:** Class-based tuple (introduced in C# 4.0).

Example:

```
Tuple<int, string, double> student = new Tuple<int, string, double>(1, "Alice", 85.5);
Console.WriteLine(student.Item2); // Output: Alice
```

Difference Between Value and Reference Tuples

Feature	Value Tuple	Reference Tuple
Introduced	C# 7.0	C# 4.0
Type	Struct (Value)	Class (Reference)
Performance	Faster	Slower
Immutability	Mutable	Immutable
Syntax Simplicity	Better	Complex

Conclusion

Use **generic collections** for type safety and performance. Use **non-generic collections** for compatibility with legacy code. Choose **value tuples** for modern, efficient data grouping and **reference tuples** for older frameworks or immutable scenarios.